Laboratory Work 2: REST API and Message Queues

Distributed and Cloud Computing | Individual laboratory report

1. Objective

The goal was to implement synchronous communication through a REST API and asynchronous communication through a message queue, then compare their behavior. The implementation uses Python, Flask, and the requests library.

2. Implementation

Part A - REST API. The Flask application api.py runs on port 5000. GET /time returns the current server time as JSON with HTTP 200. POST /echo returns the JSON message with HTTP 200; when the required message field is absent, it returns HTTP 400 with an error description.

Part B - Message queue. The Flask broker broker.py runs on port 6000. POST /queue/orders appends a JSON message to an in-memory queue and returns HTTP 201. GET /queue/orders removes and returns the oldest message (HTTP 200), or returns HTTP 204 when empty. The producer submits orders about once per second; the consumer fetches and simulates processing an order for three seconds. FIFO ordering is provided by removing the first element of the list.

3. Testing and evidence

Test / evidence file	Expected behavior
GET /time - screenshot_1.png	JSON server_time, HTTP 200
POST /echo - screenshot_2.png	JSON echoes message, HTTP 200
Invalid POST /echo - screenshot_3.png	Missing message produces HTTP 400
Producer and consumer - screenshot_4.png	Producer sends independently; consumer processes more slowly

The screenshot references above correspond to the required evidence files stored in the separate screenshots/ folder. The 30-second consumer interruption is discussed below as the expected behavior of this in-memory design.

4. Discussion questions

1. REST request vs. TCP session. In Lab 1, the application protocol used a persistent TCP connection, so multiple commands could be exchanged during the same session. In this lab, each REST operation is a separate HTTP request and response. HTTP may reuse underlying TCP connections, but the REST application does not require a persistent client session or stored conversation state.

2. Why POST for /echo? POST sends the message in the JSON request body and allows structured input. Using GET with URL parameters would expose the message in the URL and make it subject to URL length and caching considerations. GET is normally intended for retrieving a resource without request-body input; POST is the method explicitly required by this API contract.

3. Consumer stopped for 30 seconds. While producer.py keeps running, it can continue adding orders to the broker; at roughly one message per second, approximately 30 additional orders may arrive in 30 seconds. After restarting consumer.py, older queued messages are returned first. These messages are preserved only while broker.py remains running; restarting the in-memory broker loses the queue. Because messages are removed on GET, this prototype also has no acknowledgement or retry mechanism if a consumer fails after receiving a message.

4. When is REST better, and when is a queue better? REST is suitable when a client needs an immediate answer, such as checking an account balance. A message queue is more suitable for background work, such as processing incoming orders or sending email notifications, when the sender should not wait for the work to finish.

5. Conclusion

The two approaches provide different interaction patterns. REST returns results directly to the caller, while the broker decouples message production from consumption. The prototype demonstrates FIFO delivery and temporary buffering, but an in-memory queue without durable storage or acknowledgements is not appropriate for production-critical tasks.